Lab 2: Finding the maximum value

Phillip Pinter

February 29, 2024

1 Introduction

The goal of this lab session is to find the maximum value of an array using three different methods:

1. A nested for loop on the CPU: make a max value variable that starts at zero and iterate over the array using a for loop. When the current element in the for loop is greater than the max value variable, the variable is overwritten by the value of the element.

2. A GPU kernel employing atomic operation: each thread reads the element of the input array corresponding with its global thread ID. Using the atomicMax() function, the maximum value of all threads within a block is found. To find the maximum value of the entire array, the kernel function is called multiple times: first, to find the maximum value within each block and then to find the maximum value among the maximum values obtained from all blocks to find the global maximum value of the array.

3. A GPU kernel using reduction: in the beginning, each thread checks 2 elements of the array, so there are e.g. 64 threads for an array of size 128. Each threads checks which of the two elements is the biggest and puts that value in the first element. After this, half as many threads are used to do the same but then with an offset twice as big (the offset starts with value 1). So if e.g. the first thread checked in the beginning element 0 and 1, it now checks element 0 and 2 and after that it checks element 0 and 4 and so on. Only the first value of each offset is needed since each thread puts the biggest value of the 2 elements in their first element (first element of the two elements the thread is checking, not the first element of the array). This way, after each iteration, the amount of elements being compared halves until one value remains: the maximum value at the location of element 0.
   Here it is essential to make sure the threads are synchronized after each iteration since thread 0 could read element 2 prematurely when thread 1 is not done putting the biggest number of element 2 and 3 in element 2 when there is no synchronization.

To showcase the performance of the three methods, the functions are timed to see which one is faster than the others.

2 Results of comparing the methods

To showcase the differences, different array sizes were used. To get accurate results, each method was run multiple times using the same array size to find an average time for a method for that specific array size. The times to calculate the averages were all pretty consistent for arrays bigger than 1 MB, they varied no more than 2% and this is acceptable since the averages between methods vary more than that. All methods provided the correct maximum value.
With a small array size of 1024 (=1kB), the for loop on the CPU was the fastest by far, with atomic the second fastest and reduction in last place.
When a bigger array of size 1048576 (=1MB) was used, the for loop and reduction took about the same amount of time with atomic in last place but the difference between the for loop and the two kernel functions were substantially smaller.
When going even bigger to a size of 100 and 500MB, the power of parallelization really started to show. Both the kernel functions were now faster than the for loop with reduction being the fastest.